ICE TEA: Insertion of Custom Early Exits for Time-, Energy- & Anomaly-Aware Neural Networks

Matthias Stammler, Julian Hoefer, Patrick Schmidt, Tanja Harbaum and Jürgen Becker
Karlsruhe Institute of Technology (KIT), Karlsruhe, Germany
Email: {stammler, julian.hoefer, patrick.schmidt2, harbaum, becker}@kit.edu

***Abstract*—Integrating neural network inference in systems with variable execution contexts proves difficult due to the difficulty in predicting runtime and energy consumption of embedded accelerators. Additionally, out of distribution inputs, adversarial attacks or latency requirements and available power budget can lead to the necessity of ending inference early. This comes at the cost of a lower confidence in the inference result. In case of out of distribution samples, classification diverging from the original class set might be desired.**

Incorporating early exits into existing neural networks for dynamically shortening the inference proves beneficial in situations where the original model structure or parameters cannot be altered, when training data is limited or to integrate neural network models into systems otherwise unsuitable due to high execution times.

To allow the adaptation of existing models, we introduce ICE TEA, a tool implementing the insertion of custom early exits to existing neural networks to integrate them in a time- or energy constraint, or anomaly-prone context. We evaluate our framework using GoogleNet and ResNeXt-101 and were able to detect anomalies with an accuracy up to 89.1 %. Per introduced early exit, we achieve an accuracy of up to 68.3 % per exit, with an equivalence of 75.9 % to the original model.

***Index Terms*—Early Exit Insertion, Edge Inference, Anomaly Detection, Multi-Exit Neural Networks, Context Monitoring**

I. Introduction

Recent years show the emergence of deep neural network models in image segmentation and classification in the domains of smart agriculture [1], autonomous driving [2] and integration into embedded devices [3]. While deep learning methods are powerful and accurate [4], they inherently lack a method to validate their results and can be found victim to adversarial attacks [5] as well as problems resulting from out-of-distribution inputs [6], because classification is always forced, regardless of the input. This proves the need for anomaly detection methodology in parallel to execution.

In [7] a convolutional neural network is used with the complete layer traces as an input to classify input images into normal or abnormal operation, and in [8] the Mahalanobis Distance is used to distinguish normal samples from out-of-distribution or deliberate malicious samples. Both methodologies achieve a very high accuracy in detecting samples deemed anomalous or out of distribution.

Branching neural networks are used to reduce the energy consumption of edge-inference at the cost of accuracy [9], provide an opportunity for faster response time in a smart grid scenario [10]. These methods describe an integrated approach with determining the right set and position of early exits as a contentious question in designing branching neural networks or networks with early exits. These methods do not cover the addition of early exits to **existing** neural networks [11] due to limitations presenting themselves as a lack of training data or the lack of access to parameters of the network structure, resulting in the need to add early exits in contrast to altering the existing network structure.

This work was funded by the German Federal Ministry of Education and Research (BMBF) under grant number 16ME0454 (EMDRIVE). The responsibility for the content of this publication lies with the authors.

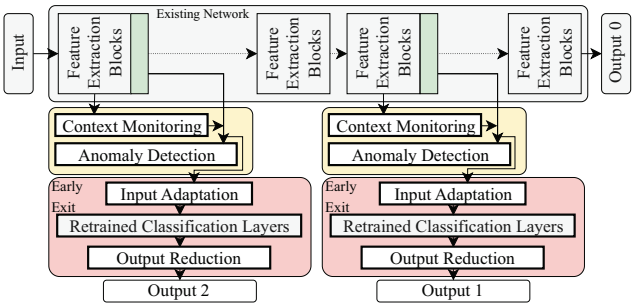

Fig. 1. Example of anomaly detection triggered early exit network. A subset (green) of all layers (green & gray) contributes to the anomaly detection (yellow). Depending on the anomaly detection output, an early exit layer (red) might be chosen. This coloring is consistent throughout the paper.

While some of these methods exist in isolation, a combination of anomaly detection and early exit addition rarely achieved.

For this, we present ICE TEA: the **I**nsertion of **C**ustom **E**arly Exits for **T**ime-, **E**nergy- and **A**nomaly aware neural networks. Our work is able to add the parts described in Fig. 1 onto existing neural network models without the need to retrain the existing model. This example is gradually built, presents the result of this work and its main contribution. This paper subsequently presents the following parts:

- A data driven methodology to determine promising layers for early exit placement, as well as training of early exits under consideration of reduced complexity (Section II-A).
- Introducing a time-, energy- and anomaly aware detection methodology to determine the usage of inserted trained early exits (Section II-B).
- Combining these methods (Section II-C) and evaluating them for GoogleNet and ResNeXt-101 (Section III).

2024 IEEE Computer Society Annual Symposium on VLSI (ISVLSI) | 979-8-3503-5411-9/24/$31.00 ©2024 IEEE | DOI: 10.1109/ISVLSI61997.2024.00125

2159-3477/24/$31.00 ©2024 IEEE
DOI 10.1109/ISVLSI61997.2024.00125

TABLE I
MODEL VARIABLES, THEIR DESCRIPTION, AND USED SECTION (SEC.)

Sec.	Variable	Description
II-A, II-B	L, L_A, L_B	All / monitored / branching layers
	$C, \widehat{C^p}$	Classes (of abstraction lvl. p)
II-A	$\widehat{t}(l), \widehat{e}(l)$	Latency & energy for $\{l_i \in L \vert l_i \leqslant l\}$
	$t_0(l), e_0(l)$	Exp. latency & energy for l
	$\breve{t}(l), \breve{e}(l)$	Exp. latency & energy for $\{l_i \in L \vert l < l_i\}$
	$t_{\max}, e_{\max}$	Time & energy budget
	$F_{\text{cost}}, w, \sigma$	Cost function, buffer window, safety factor
II-B	$b_a : L \to \mathbb{N}$	Anomaly severity
	$b_c : L \to \{0; 1\}$	Context status

II. METHODOLOGY

The tool flow of this work is divided into the placement and training of early exits (Section II-A), the placement and training of anomaly detection methods (Section II-B) and the system integration (Section II-C). The following notation is used in the rest of this paper.

We define the set of all neural network layers as $L = \{l_n \mid n \in [1, N]\}$, where N is the number of all neural network layers. We further define an ordering $<$ over L as, if one layer $l_x \in L$ is executed before $l_y \in L$, then $l_x < l_y$.

We define the set of all neural network layers used for anomaly detection or time and energy monitoring $L_A \subseteq L$ as $L_A = \{l_{a,n} \in L \mid n \in [1, N_A]\}$, where $N_A = |L_A|$ is the total amount of neural network layers used for anomaly detection. We furthermore define $L_B \subset L$ as the set of all neural network layers facilitating a branch into an early exit. L_B is built similarly to L_A as $L_B = \{l_{b,n} \in L \mid n \in [1, N_B]\}$, where $N_B = |L_B|$ is the total amount of added early exits. Both sets L_A and L_B do not need to be disjoint. A layer can both be considered for anomaly detection and branching into an early exit.

We furthermore define all possible input classes the neural network can distinguish as $C = \{c_i \mid i \in [1, N_C]\}$, where $N_C = |C|$ is the total amount of all possible classes. A class set signifying a higher level of abstraction is denoted as $\widehat{C^p}$ and defined as $\widehat{C^p} = \{\widehat{c_i^p} \mid i \in [1, N_C^p]\}$, where $N_C^p = |\widehat{C^p}|$ signifies the number of classes in this class set. The superscript p denotes the corresponding abstraction level, where a higher number signifies a higher abstraction level and $C = \widehat{C^0}$ represents the original class set with the least abstraction.

A. Early Exit Addition

Using an introduced early exit is either triggered by hitting a time or energy budget constraint or by the detection of an anomaly. Because of this, the placement and training of these early exits needs to consider the given constraints.

For this, we define $t_0 : L \to \mathbb{R}^+$ and $e_0 : L \to \mathbb{R}^+$ as the expected runtime and energy consumption for each layer. Additionally, we define $t_{\max} \in \mathbb{R}^+$ and $e_{\max} \in \mathbb{R}^+$ as the maximum runtime and energy budget for the whole network.

Lastly, the sum of all execution time $\widehat{t} : L \to \mathbb{R}^+$ and energy consumption $\widehat{e} : L \to \mathbb{R}^+$ of all layers until and including a given layer $l_x \in L$ is calculated as:

$$\widehat{t}(l_x) = \sum_{l \in L}^{l \leqslant l_x} t(l) \qquad \widehat{e}(l_x) = \sum_{l \in L}^{l \leqslant l_x} e(l),$$

with measured execution time of one layer denoted as $t : L \to \mathbb{R}^+$ and the measured energy denoted as $e : L \to \mathbb{R}^+$. We define $\breve{t} : L \to \mathbb{R}^+$ and $\breve{e} : L \to \mathbb{R}^+$ as the expected execution time and energy consumption for the rest of the inference after a given layer $l_x \in L$. This is calculated straightforwardly as:

$$\breve{t}(l_x) = \sum_{l \in L}^{l > l_x} t_0(l) \qquad \breve{e}(l_x) = \sum_{l \in L}^{l > l_x} e_0(l),$$

where $t_0(l)$ and $e_0(l)$ are the expected runtime and energy budget.

Choosing the right position to place an early exit and trace based anomaly detection significantly expands the design space. For this reason, we divide the problem into a placement and a separate training step, both explained below.

While the placement of all early exits is important, at least on early exit needs to guarantee the execution under a given maximum execution time as well as the usage of less than the maximum energy budget. To achieve this, at least one selected layer $l \in L$ must lie within an expected execution time and energy budget of less than the maximum divided by a safety factor σ_t and σ_e.

Furthermore, each layer $l_b \in L_B$ is chosen to minimize a cost function F_{cost} within a given region. For the first branching layer $l_{b,1} \in L_B$, this equals to

$$l_{b,1} = \arg\min\{F_{\text{cost}}(l) \vert l \in \{l_1, ..., l_m\}\},$$

where $\{l_1, ..., l_m\} \subset L$, and l_m is the last layer, adhering to $\sigma_t \cdot \widehat{t}(l_m) < e_{\max}$ and $\sigma_e \cdot \widehat{e}(l_m) < e_{\max}$. Every subsequent branching layer is then calculated as

$$l_{b,i} = \arg\min\{F_{\text{cost}}(l) \vert l \in \{l_m, ..., l_n\}\},$$

with the index m being equal to the index of the previous branching layer plus *half* of a buffer window w and the index n being equal to the index of the previous branching layer plus *one and a half* of the buffer window w. This window is introduced to allow for an even spacing of early exits in large models.

With this introduced buffer window, this results in two early exits to stay within a given width of at least $w/2$ and at most $3w/2$. The input of each early exit branch with index k is given as the output of the preceding layer $l_{b,k} \in L_B$ of the original network. This allows us to design each classifier based on the collection of *layer traces* generated by layer $l_{b,k} \in L_B$, reducing the need for multiple inference runs per input image. By generating *layer traces* for the whole set of early exit layers L_B during one inference, one traced inference over the

complete training dataset generates the input data for every early exit classifier.

Using the EFFECT tool [12] to evaluate classification strategies for each early exit classifier independently allows for an automated selection of applicable classification strategy with combined training. After an independent run of EFFECT for each layer $l_{b,k} \in L_B$, each early exit is configured and ready for use. To stay as close to the original classifier of the given network as possible, the structure of the classifier mimics the structure of the classifier of the original network after $l_{b,N} \in L$ with $N = |L|$, allowing for configured accelerators to be used efficiently for all (early) exits.

This results in the red structure depicted in Fig. 1, where after a subset of feature extraction blocks (green) of the existing network, an early exit (red) is placed.

B. System Monitoring

System monitoring during inference splits into two distinct parts. We define the first part of continuous monitoring of time and energy as *Context Monitoring* and the monitoring for inference anomalies as *Anomaly Detection*.

Similar to the insertion of early exits described in Section II-A, the selection of monitored layers is equivalent to choosing the right layers for $L_A^c \subseteq L$ and configuring them. Fig. 1 shows the desired structure, where the monitoring (yellow) is split into two parts and controls the chosen early exit (red). Choosing layers for *Context monitoring* is trivial in our tool. For this, a layer is considered and entered in L_A^c, if it lies before an early exit, resulting in $L_A^c \supseteq L_B$. At the point of choosing an early exit, the remaining time and energy budget is compared against the measured runtime and energy consumption, as well as expected runtime and consumption budget. We define $b_c : L_B \rightarrow \{0; 1\}$ as to signal, if the corresponding early exit is chosen because of the execution context. If an early exit is chosen, then $b_c = 1$, else $b_c = 0$.

Configuring anomaly detection components is itself split into two parts, the placement of anomaly detection classifiers as well as the parameterization and training of these classifiers. The placement is in this case equal to adding to the set L_A^a, by determining suitable anomaly detection points. Both steps are similar to determining early exits as described in Section II-A. Because the marking of at least one layer for anomaly detection between every early exit is necessary to ensure the possibility of choosing the respective early exit layer in case an anomaly is detected, at least one layer $l_{a,k} \in L$ must be marked as an anomaly detection layer between two layers marked for early exiting.

Determining the suitable anomaly detection layers between two early exit layers also depends on a cost function $F_{\text{cost}} : L \rightarrow \mathbb{R}^+$. Utilizing this cost function, we try to minimize it over the set of neural network layers situated between two early exits which is given by the set $L_{C,i}$

$$L_{C,i} = \{l_c \in L \mid l_i < l_c \leqslant l_{i+1}, l_i \in L_B, l_{i+1} \in L_B\}.$$

The layer, resulting in a cost function minimum of each set $L_{C,i}$ is then collected and added to L_A^a.

$$L_A^a = \{l_i \in L \mid l_i = \underset{l_c \in L_{C,i}}{\arg\min} \{F_{\text{cost}}(l_c)\}$$

After determining the set of anomaly detection layers L_A^a, parameterization and training of anomaly detection classifiers takes place by utilizing the EFFECT framework [12]. In this work, we determine the best classifier, for classifying an inference into different levels between normal operation and anomalies resulting in the need for inference abortion. The detection of an anomaly is denoted by the function $b_a : L_A^a \rightarrow \mathbb{N}$, where $b_a(l) \neq 0$ signifies a detection of an anomaly by the corresponding classifier. The resulting action is determined by the anomaly severity and can range between continued inference and complete abortion.

Anomaly Detection layers and *Context Monitoring* layers are combined into $L_A = L_A^c \cup L_A^a$.

C. System Integration

The resulting system itself is split into two parts, the *System Configuration* and the *Execution Environment*. Both parts are implemented using the PyTorch framework version 1.13.1+cu117 [13] running on an NVIDIA RTX A6000 graphics card with CUDA version 12.3.

The combination workflow starts with one complete and traced inference of each input sample. For each inference, execution time, energy consumption and layer traces are collected in parallel. Over a training dataset, the layer traces of each layer l_k potentially in L_A or L_B are collected. In parallel, the runtime $t(l)$ and energy consumption $e(l)$ of each layer $l \in L$ is measured for each input datum and added to a collection, respectively. Afterward, we estimate the expected maximum execution time for each layer $l_a \in L_A$ as the maximum of all monitored execution times. The maximum expected energy consumption is calculated similarly.

Using the traces from each traced layer l_k, the cost function F_{cost} is evaluated for determining L_A and L_B. Trivial reasoning suggests that a chosen measurement must be small between the samples of one class and large between two samples of different classes, reflecting the possibility of a high classification accuracy and a high chance for anomaly detection. The cost function thus describes the sum of an average distance metric between all samples of a class $c_i \in C$ and all other classes $c_\lambda \in C \backslash \{c_i\}$. While other cost functions might be applicable, we chose this for its triviality.

For this distance measurement, we choose the cross-entropy, resulting in the cost function representing an inter-class entropy. As a measurement for information gain, this estimates the ability to discriminate between classes in the traces of the chosen layer in contrast to a classification after other not chosen layers.

With the first layer $l_0 \in L_B$ being constraint by the time and energy budget and the subsequent layers determined by the window w and the local minimum of F_{cost}, the set L_B is built. Because an early exit is preceded by a lower amount of

TABLE II
ACCURACY & EQUIVALENCE BETWEEN ORIGINAL AND EARLY EXIT

Network	$\widehat{C^0}$	$\widehat{C^1}$		$\widehat{C^2}$		$\widehat{C^3}$	
	Acc.	Acc.	Eq.	Acc.	Eq.	Acc.	Eq.
GN. layer		Inception 6		Inception 5		Inception 4	
Early Exit	/	63.6	69.9	64.1	72.0	64.2	74.1
Baseline	69.8	74.1	/	76.1	/	77.4	/
RN. layer		Residual 24		Residual 17		Residual 9	
Early Exit	/	64.7	74.0	67.2	75.8	68.3	75.9
Baseline	80.2	83.6	/	85.2	/	86.0	/

TABLE III
ACCURACY FOR ANOMALY DETECTION METHODS IN PERCENT

Network	Layer $L_{A,A}$	(1) Eucl.	(2) Maha.	(3) NN.
GoogleNet [14]	Inception 4	78.0	81.5	84.9
	Inception 5	78.2	83.8	87.7
	Inception 6	80.9	83.9	89.1
ResNeXt101 [15]	Residual 9	76.3	83.0	83.4
	Residual 17	78.7	85.7	87.7
	Residual 24	82.0	85.9	88.9

feature extraction than the original classifier, the classification granularity is reduced for each early exit. In our model, this is signified by the reduced class set $\widehat{C^p}$ with $p = N_B - k$, where p is the abstraction level, $N_B = |L_B|$ signifies the amount of chosen layers to add an early exit to and k is the index of the current layer $l_{b,k} \in L_B$.

For the training of anomaly detection methods, the anomaly classes are chosen to be $C = \{\texttt{norm}, \texttt{anomaly}, \texttt{cancel}\}$. The first classification `normal` results in no change in execution ($b_a = 0$), the second classification `anomaly` ($b_a = 1$) leads to choosing an early exit and the last classification `cancel` ($b_a = 2$) leads to an immediate abortion of inference.

After early exits as well as anomaly detection and context monitoring are configured and added to the system, execution of early exits is implemented using the PyTorch layer hook system. If a layer $l_a \in L_A$ is executed, the anomaly detection classifies the corresponding trace. For each layer $l \in L$, the execution time $t(l)$ and energy consumption $e(l)$ is logged. At every possible branching point $l_b \in L_B$, the corresponding early exit is executed if $b_c(l_b) = 1 \vee b_a(l_a) = 1$. If $b_a(l_{b,k}) = 2$, inference is aborted. Otherwise, inference continues with the original network.

III. EVALUATION

We evaluate our approach by using ResNeXt-101 [15] and GoogLeNet [14] trained on ImageNet-1k [17] as a case study. For both, we add $N = 3$ early exits to the network. For a simple and comprehensible structure and the potential to use similar hardware resources, every early exit follows the same structure. This structure starts with an adaptation layer towards the original network, designed for dimension matching. Afterward, a classification layer with the same structure as the original network's classification layer is placed. In GoogleNet these follow the structure of *Inception Layers*, in ResNeXt-101, these follow the structure of *Residual Blocks*. The training of each early exit is facilitated by using traces of the corresponding ImageNet-1k dataset [17].

An overview of the relative measured entropy can be found in Fig. 2 as the blue bar chart and is calculated with 5% of randomly chosen samples of each class. The highest entropy apart from the output layer marked at 100 %. The selected layers L_B are filled in blue. The last possible layer for the first branching point $l_{b,0} \in L_B$ is considered as $0.5 \cdot \widehat{t}(l)$ and is marked cyan.

The execution time and energy consumption is measured for every layer, while generating traces for entropy calculation based on the ImageNet-1k validation dataset. After L_A is built, the expected execution time is calculated and summed up for every layer. Fig. 2 shows the maximum expected execution time $\widehat{t}(l)$ until every layer as the red line. For GoogleNet, the subsequent *Inception Layers* 4, 5 and 6 are chosen as an early exit start. For ResNeXt-101, *Residual Block* 9 shows the highest inter-class entropy, while 17 is a local optimum and falls directly onto the cutoff point.

To accommodate for the expected drop in accuracy due to a reduced amount of feature extraction, we reduce the number of classes for each early exit, leading to $|\widehat{C^{p+1}}| < |\widehat{C^p}|$, where $\widehat{C^p}$ signifies the classes used for the early exit starting from $l_{b,k} \in L_B$, with $p = N_B - k$ and $p = 0$ signifying the original classification layer as well as $p = N_B - 1$ the earliest exit.

In this work, the original ImageNet-1k data set represents $\widehat{C^0} = C$. Because ImageNet samples are classified based on WordNet3 [18] synsets [17], a reduction in classes is achieved by merging synsets into their nearest hypernym. If multiple hypernyms exist per synset, the nearest hypernym is chosen based on the `path_similarity` distance. This results in a clustering of sysnsets into the respective hypernyms without overlaps in semantic meaning, resulting in $|\widehat{C^0}| = 1000$, $|\widehat{C^1}| = 557$, $|\widehat{C^2}| = 304$ and $|\widehat{C^3}| = 184$. We evaluate the system in terms of accuracy and equivalence to the original model.

For each introduced early exit layer accuracy and equivalence to the original model is traced and depicted in Table II. The accuracy slightly increases with increased levels of abstraction, as does the equivalence to the original model, considering the reduced complexity of the dataset. In all cases, the accuracy falls below the baseline of the original model, with the highest accuracy and equivalence for *Residual Blocks* 9 in ResNeXt-101.

To evaluate the anomaly detection capabilities, we differentiate between three anomaly classes, $C = \{\texttt{norm}, \texttt{anomaly}, \texttt{cancel}\}$. A trace is classified as `normal`, if it is in ImageNet-1k [17]. If a trace is generated from ImageNet-A [19] or ImageNet-R [20], it is classified as `anomaly`. Lastly, if a trace resembles an out of distribution sample from ImageNet-O [19], or a white noise input, it is classified as `cancel`. We compare three

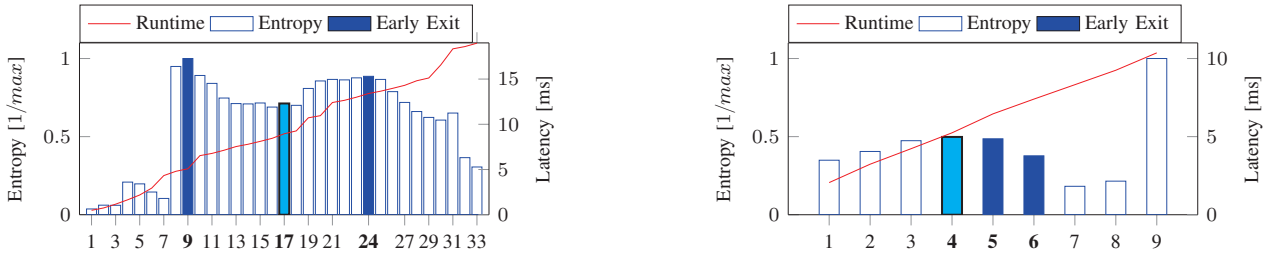

Fig. 2. Entropy values & runtime for Residual Blocks for ResNeXt-101 [16] (left) and Inception layers for GoogleNet [14] (right). Red shows the runtime until after the layer, the relative entropy is given outlined in blue. Filled bars signify the start of early exits. Cyan signifies the latest possible first early exit layer.

different anomaly detection approaches, (1) using *Euclidean distance* between samples and the class average, (2) using the *Mahalanobis distance* and (3) training a convolutional *neural network classifier*. The accuracy of the anomaly detection methods are depicted in Table III. In all cases, the anomaly detection accuracy increases towards the end of the network. The neural network approach is in all cases the most accurate in determining anomalies, reaching up to 89.1 % for GoogleNet *Inception Layer* 6.

IV. Conclusion

In this work, we present ICE TEA, a framework to automatically add early exits onto existing neural networks, without the need for retraining or altering of network structures. The methodology includes the placement and automated training of early exit to classify inputs into classes with decreasing abstraction, in case inference needs to be finished early. Accompanied by a context monitoring system, the adapted neural network continues unaltered inference in contexts, not requiring alteration. If the context monitoring or anomaly detection system signals the need for choosing an early exit, the early exit is executed using the outputs from the corresponding network layer.

Applicable in cases where the parameters of the original network are not available, our approach facilitates the dynamic shortening of the inference while staying close to the original model architecture. Additionally, the reuse of the original output layer structures aides in the mapping to existing accelerator hardware and the integration into an embedded processing system. The framework is validated for GoogleNet and ResNeXt-101, by adding 3 early exits to both networks each. Because ImageNet classes are equivalent to WordNet synsets, the abstraction of output classes is facilitated by finding the nearest hypernym for every class. With an acceptable reduction in accuracy, the inference using early exits designed with our approach has a near constant equivalence to the original neural network models.

References

[1] A. Escamilla-García *et al.*, "Applications of artificial neural networks in greenhouse technology and overview for smart agriculture development," *Applied Sciences*, vol. 10, no. 11, p. 3835, 2020.

[2] T.-D. Do *et al.*, "Real-time self-driving car navigation using deep neural network," in *2018 4th International Conference on Green Technology and Sustainable Development (GTSD)*, pp. 7–12, IEEE, 2018.

[3] Y. S. S. et al., "Simba: Scaling deep-learning inference with multi-chip-module-based architecture," in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, MICRO '52, (NY, NY, USA), p. 14–27, Association for Computing Machinery, 2019.

[4] S. Smys, J. I. Z. Chen, and S. Shakya, "Survey on neural network architectures with deep learning," *Journal of Soft Computing Paradigm (JSCP)*, vol. 2, no. 03, pp. 186–194, 2020.

[5] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," *arXiv preprint arXiv:1412.6572*, 2014.

[6] D. Hendrycks and K. Gimpel, "A baseline for detecting misclassified and out-of-distribution examples in neural networks," *arXiv preprint arXiv:1610.02136*, 2016.

[7] C. Schorn and L. Gauerhof, "Facer: A universal framework for detecting anomalous operation of deep neural networks," in *2020 IEEE 23rd International Conference on Intelligent Transportation Systems (ITSC)*, pp. 1–6, 2020.

[8] Y.-C. Hsu *et al.*, "Generalized ODIN: Detecting out-of-distribution image without learning from out-of-distribution data," in *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, IEEE, 2020.

[9] Y. Wu *et al.*, "Intermittent inference with nonuniformly compressed multi-exit neural network for energy harvesting powered devices," in *2020 57th ACM/IEEE Design Automation Conference (DAC)*, pp. 1–6, 2020.

[10] A. Vashist *et al.*, "Dqn based exit selection in multi-exit deep neural networks for applications targeting situation awareness," in *2022 IEEE International Conference on Consumer Electronics (ICCE)*, pp. 1–6, 2022.

[11] S. Scardapane *et al.*, "Why should we add early exits to neural networks?," *Cognitive Computation*, vol. 12, no. 5, pp. 954–966, 2020.

[12] M. Stammler *et al.*, "Effect: An end-to-end framework for evaluating strategies for parallel ai anomaly detection," *Procedia Computer Science*, vol. 222, pp. 499–508, 2023.

[13] A. P. et al., "Pytorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems 32* (H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, eds.), pp. 8024–8035, Curran Associates, Inc., 2019.

[14] C. Szegedy *et al.*, "Going deeper with convolutions," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 1–9, 2015.

[15] S. Xie *et al.*, "Aggregated residual transformations for deep neural networks," *CoRR*, vol. abs/1611.05431, 2016.

[16] X. Xie and K.-H. Kim, "Source compression with bounded dnn perception loss for iot edge computer vision," in *The 25th Annual International Conference on Mobile Computing and Networking*, MobiCom '19, (NY, NY, USA), Association for Computing Machinery, 2019.

[17] J. Deng *et al.*, "Imagenet: A large-scale hierarchical image database," in *2009 IEEE Conference on Computer Vision and Pattern Recognition*, pp. 248–255, 2009.

[18] C. Fellbaum, "Wordnet," in *Theory and applications of ontology: computer applications*, pp. 231–243, Springer, 2010.

[19] D. Hendrycks *et al.*, "Natural adversarial examples," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 15262–15271, 2021.

[20] D. Hendrycks *et al.*, "The many faces of robustness: A critical analysis of out-of-distribution generalization," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 8340–8349, 2021.